Cute-YOLO Studio

User Manual

Train, adapt, quantize, package, and deploy a fixed single-class detector over BLE

Fixed architecture: cute_yolo_fixed_dualcore_hybrid_8_24
Input: 128 x 128 grayscale
Detection head: 16 x 16 x 5 [objectness, dx, dy, w, h]
Trainable parameters: 23,925 | Convolution MACs: 6,995,968

August 2026

1. Quick Start

Cute-YOLO Studio organizes the complete workflow into four tabs. A typical project moves from source-domain training to optional real-domain adaptation, then packages the INT8 detector as a .cute file and uploads it to an ESP32 over BLE.

TRAIN -> ADAPT TO REAL DATA -> DEPLOY VIA BLE -> VERIFY IN LOG

First-time users: start with one of the ready-to-use ZIPs or download notes in dataset_examples/.

1. Prepare a single-class YOLO-format ZIP for base training.
2. Train the fixed Cute-YOLO model and optionally create the base .cute package.
3. If camera-domain performance needs improvement, collect and label real ESP32 images with collect_cute_dataset.py and cute_label_gui.py, then run domain adaptation.
4. Deploy the base or adapted .cute package over BLE.
5. Confirm transfer, validation, and activation in the Log tab.

Paths: All paths shown in the screenshots are examples. Your dataset, output, and model paths can be different.

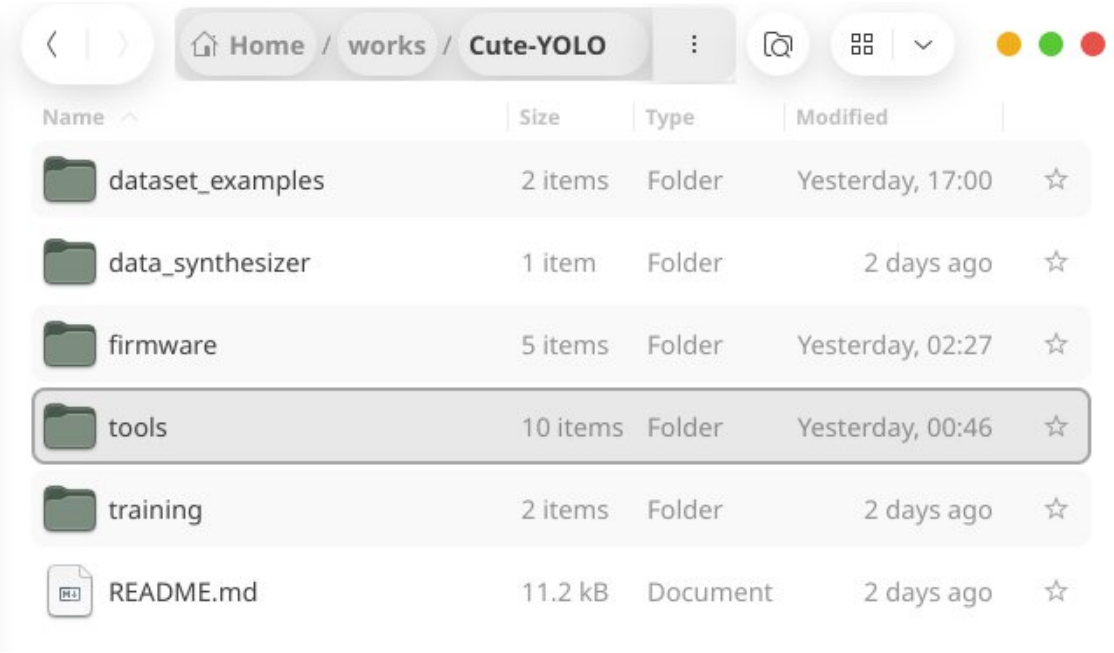
Recommended starting recipes

Object type	Preset	Min. size	Overlap loss
Faces / vehicles / animals	Complex / distinctive	8 px is a useful starting point	CIoU
Circles / blobs / repeated shapes	Primitive / repetitive	0 px or task-dependent	CIoU

Loss profiles: The Complex preset emphasizes hard-negative mining. The Primitive preset can add box-footprint, halo, and crowd-aware negative weighting to suppress geometric pseudo-objects.

2. Repository Layout and Example Datasets

The repository is organized so a new user can move from example data to training, adaptation, firmware, and documentation without searching through a monolithic folder. The dataset_examples/ folder should demonstrate the exact ZIP formats accepted by Cute-YOLO Studio.



Repository view. A docs/ folder is recommended for the manual and project documentation.

Recommended top-level roles

Folder / file	Role in the Cute-YOLO workflow
dataset_examples/	Ready-to-use example ZIPs and dataset download notes for base training and domain adaptation.
data_synthesizer/	Synthetic-data generators, including the round-object generator used for primitive/repetitive experiments.
firmware/	ESP32 Cute-YOLO firmware, fixed INT8 inference runtime, BLE model receiver, camera/TFT integration, and device-side deployment support. See Section 4 for the current ST7735 wiring.
tools/	Cute-YOLO Studio and supporting utilities for real-domain collection, label verification, .cute packaging, and BLE upload. When the tools are run from this directory, several generated output folders also appear here.
training/	Standalone or reference training material, notebooks/scripts, and experiment-specific training support.
docs/	Recommended folder for this user manual, figures, and other project documentation.
README.md	Repository entry point: installation, quick start, hardware notes, and links to examples/documentation.

Example datasets

Use source dataset for the data used in base training, and real deployment-domain dataset for the smaller data collected from the target ESP32 camera setup and used during domain adaptation.

Round-object example

- round_synthetic.zip -> base training with the Primitive / repetitive preset.
- round_real.zip -> domain adaptation using real camera images of the same target class.

Face example

- WIDER FACE YOLO-format source dataset -> base training with the Complex / distinctive preset. Prefer a README/download note instead of committing the entire third-party dataset when redistribution or repository size is a concern.
- esp32_face_real.zip -> real deployment-domain data exported by the Cute-YOLO labeling workflow and used for adaptation.

Expected single-class YOLO ZIP organization

```
dataset.zip
  images/
    image001.jpg
    image002.jpg
  labels/
    image001.txt
    image002.txt
  classes.txt          (optional; otherwise the Studio uses class "X")

Each label row:  0  x_center  y_center  width  height
Coordinates are normalized to [0, 1].
```

Workflow rule: source ZIP -> Tab 1 Base Training; real deployment-domain ZIP -> Tab 2 Domain Adaptation.

3. Tools Directory and Generated Folders

The tools/ directory contains both source utilities and working/output folders. Some folders are created automatically when a tool is run from tools/, so they should not be confused with hand-written source code.

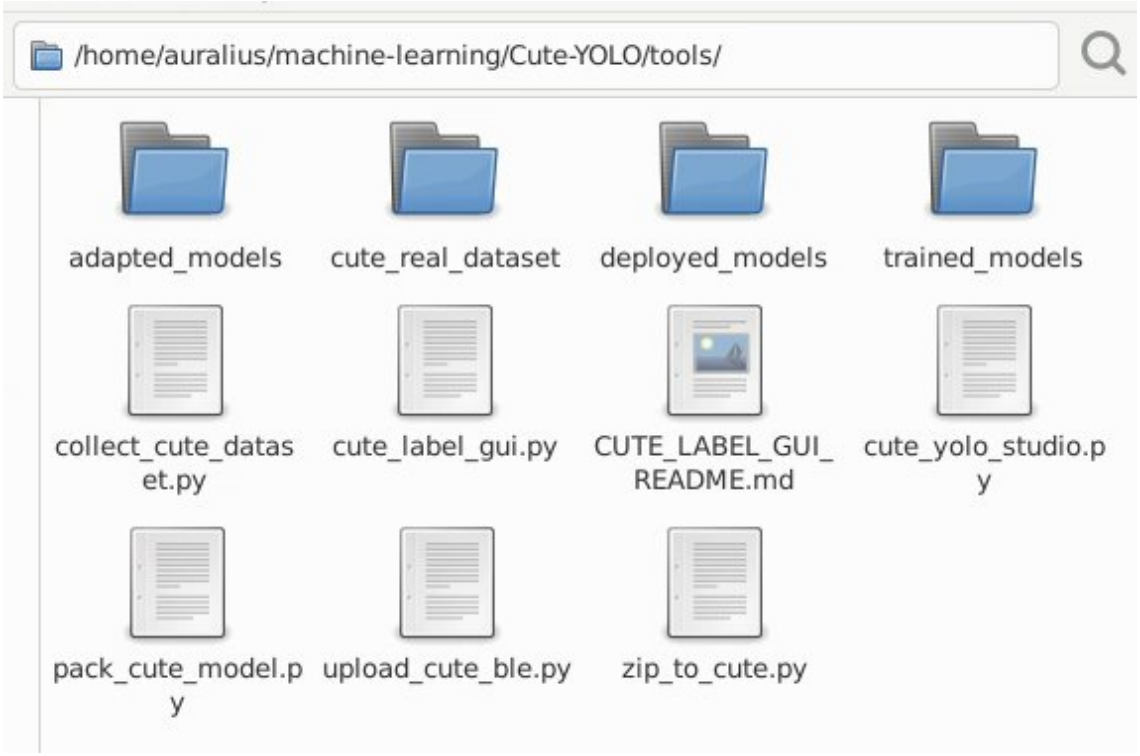


Figure 1a. tools/ directory showing source utilities together with generated training, adaptation, and collection folders.

Source utilities

The table below summarizes the current utilities and the files or folders they create.

Tool	Purpose	Creates / writes
cute_yolo_studio.py	Main four-tab desktop application for base training, domain adaptation, direct BLE deployment, and logging.	By default, Train writes to trained_models/ and Adapt writes to adapted_models/ when the Studio is launched from tools/. Each run also creates the corresponding export ZIP, debug folder, and optional .cute package. The current Deploy tab uploads an existing .cute directly and does not create a deployment folder.
collect_cute_dataset.py	Collects the exact 128 x 128 GRAY8 detector input and current ESP32 detections over USB serial for real-domain data acquisition.	Default output: cute_real_dataset/. It creates images/ (clean detector inputs), labeled/ (preview images), pseudo_labels/ (model-predicted YOLO boxes), and metadata/ (JSON confidence/timing/model information). Pseudo-labels are not verified ground truth.
cute_label_gui.py	Reviews and corrects the collector pseudo-labels, or labels images from scratch, for the single Cute-YOLO class.	Inside the selected dataset it creates/updates classes.txt, labels/, and reviewed/. Export ZIP writes a standard single-class YOLO ZIP containing classes.txt, images/, and labels/.
zip_to_cute.py	User-facing converter from a Cute-YOLO TensorFlow export ZIP to a memory-mappable .cute package.	Writes one .cute file. Train/Adapt in Cute-YOLO Studio use this converter when "Also create <class>_base.cute" or "..._adapted.cute" is enabled.

Tool	Purpose	Creates / writes
	Supports optional deployment-setting overrides.	
pack_cute_model.py	Lower-level fixed-format packer for the same 8+24 architecture. Useful for scripted/explicit packaging and format validation.	Writes the explicitly requested --out .cute file. It does not create a fixed working directory.
upload_cute_ble.py	Standalone BLE uploader for an already-built .cute package. It scans for Cute-YOLO unless --address is supplied, uploads the blob, then waits for firmware validation/activation.	No model-output folder. The file is transmitted to the ESP32 A/B model partitions; progress and status are printed to the terminal.
CUTE_LABEL_GUI_README.md	Short usage notes for the labeling GUI.	Documentation only; it does not create data.

Generated folder lifecycle

```
tools/
  collect_cute_dataset.py
  cute_label_gui.py
  cute_yolo_studio.py
  zip_to_cute.py
  pack_cute_model.py
  upload_cute_ble.py
  CUTE_LABEL_GUI_README.md
  cute_real_dataset/    <- generated by collect_cute_dataset.py
  trained_models/       <- default Studio Train output
  adapted_models/       <- default Studio Adapt output
  deployed_models/      <- legacy/optional folder from an older Studio flow
```

Important: in the current four-tab Studio, Deploy via BLE consumes an existing .cute file directly. Therefore deployed_models/ is not required by the current deployment tab and can be treated as a legacy or user-managed folder if it remains in the repository working tree. Generated folders such as trained_models/, adapted_models/, and cute_real_dataset/ are normally working artifacts; keep curated public samples under dataset_examples/ and consider excluding generated working folders from version control.

Real-domain data workflow

The collection and labeling utilities form a small pipeline before domain adaptation:

```
ESP32 camera + active .cute
  -> collect_cute_dataset.py
  -> cute_real_dataset/
       images/ + pseudo_labels/ + metadata/
  -> cute_label_gui.py
       verify/correct boxes -> labels/
  -> Export ZIP
  -> Cute-YOLO Studio: Adapt to Real Data
```

4. ESP32-S3 Hardware Wiring

The current fixed 8+24 Cute-YOLO firmware uses an ST7735 128 x 160 SPI TFT. The display is initialized in landscape orientation, giving a 160 x 128 screen. The camera wiring is defined separately by camera_pins.h; the table below covers the TFT interface used by the detector firmware.

TFT label	ESP32-S3 pin	Function / note
SCK / SCL / CLK	GPIO39	SPI clock
SDA / MOSI / DIN	GPIO40	SPI MOSI (display data). On many ST7735 modules this pin is labeled SDA even though the interface is SPI.
CS	GPIO38	SPI chip select
DC / A0 / RS	GPIO41	Data / command select
RST / RES	GPIO42	Display reset
MISO	Not connected	The firmware sets TFT_MISO = -1; the display is write-only in this configuration.
GND	GND	Common ground
VCC / LED / BL	Module-dependent	Use the supply/backlight voltage specified for the particular ST7735 breakout. Do not infer the power voltage from the GPIO mapping.

Current control button: the fixed 8+24 firmware uses the ESP32-S3 DevKitC-1 built-in BOOT button on GPIO0. No extra button is required for the current firmware when that board is used.

Relevant firmware definitions:

```
#define TFT_SCLK 39
#define TFT_MOSI 40
#define TFT_CS   38
#define TFT_DC   41
#define TFT_RST  42
#define TFT_MISO -1
#define BUTTON_PIN 0
```

Display initialization:

```
SPI.begin(TFT_SCLK, TFT_MISO, TFT_MOSI, TFT_CS);
tft.initR(INITR_BLACKTAB);
tft.setRotation(1);
```

5. Tab 1 - Train

Use the Train tab to learn the complete fixed network from a source dataset. All layers are trainable during base training: the three stems, Hybrid A, Hybrid B, and the detection head.

Figure 1. Train tab. Numbered markers correspond to the explanations below.

1	Dataset ZIP and output folder. The ZIP must contain a single-class YOLO dataset. If classes.txt is absent, the Studio names the class "X".
2	Core training settings. Validation % controls the hold-out split; epochs, learning rate, and batch size control optimization.
3	Training-only augmentation. Horizontal flip is label-safe: x_center is transformed to 1 - x_center and the 16 x 16 target is re-encoded. Brightness and contrast alter only image appearance. Shuffle changes sample order each epoch.
4	Object characteristics / loss preset. Choose Complex / distinctive for rich objects such as faces, or Primitive / repetitive for repeated simple geometry. Custom exposes the individual switches.
5	Detection-loss details. Hard-negative mining focuses on confusing background cells. Minimum object size filters tiny boxes at 128 x 128. Box overlap can use IoU or CIoU.
6	Package/output controls. Enable "Also create <class>_base.cute" for a deployment-ready package, then press Train base model.

6. Understanding the Training Loss

The detector predicts a 16 x 16 grid. Each cell has five outputs: objectness plus four bounding-box quantities [dx, dy, w, h]. Only the cell containing an object center receives objectness target 1; background cells have target 0.

```
L = L_obj + 5 L_box + 2 L_overlap
Complex / distinctive:  L_obj = L_pos + 1.5 L_hard
Primitive / repetitive: task-dependent footprint / halo / crowd terms may be added
```

What the terms mean

- L_pos: binary-cross-entropy objectness loss at true object-center cells.
- L_hard: binary-cross-entropy loss over the hardest background cells - the cells most strongly mistaken for objects.
- L_box: coordinate regression loss for [dx, dy, w, h] at positive cells.
- L_overlap: IoU or CIoU loss on decoded predicted and ground-truth boxes.
- Footprint / halo negatives: specially weighted background cells around a ground-truth box. Their objectness target is still 0.
- Crowd-aware weighting: increases supervision where several nearby primitive-object footprints overlap or interact.

Grid resolution: The loss is evaluated on the 16 x 16 detection representation. Box geometry is decoded back into normalized image coordinates for IoU/CIoU. One detector cell corresponds to approximately 8 x 8 input pixels.

Training outputs

- <class>_base_export.zip - model/export data used by the Studio pipeline.
- <class>_base_debug/ - debug images and training diagnostics.
- <class>_base.cute - deployment package when the option is enabled.

Deployment calibration: After INT8 conversion, Cute-YOLO Studio selects the deployment confidence and NMS operating point on INT8 validation outputs rather than blindly copying the FP32 threshold.

7. Tab 2 - Adapt to Real Data

Domain adaptation fine-tunes a trained base detector using a mixture of the original source data and verified real deployment-domain samples. This is intended for cases where the source-validation model is good but the physical ESP32 camera domain differs in lighting, optics, scale, or background statistics.

Figure 2. Adapt to Real Data tab.

1	Input files. Select the base model export, the original source training ZIP, the real labeled ZIP, and an output folder. The real labeled ZIP is produced by the Cute-YOLO labeling workflow.
2	Mixture and optimizer settings. Original data % and Real data % control the source/target mix. Real validation % reserves target-domain samples for evaluation. Adaptation normally uses a smaller learning rate than base training.
3	Dual-branch specialized fine-tuning. When enabled, Stem 1/2/3 and the head are frozen. Hybrid A is localization-dominant; Hybrid B is objectness-dominant. The cross-loss weight keeps each branch aware of the complementary objective.
4	Training-only augmentation is available again during adaptation. Flip labels are re-encoded exactly as in base training.
5	Use the same task-appropriate loss preset and overlap loss unless you are intentionally running an ablation.
6	Enable "Also create <class>_adapted.cute" and press Adapt model to produce the target-domain package.

8. Tab 3 - Deploy via BLE

The Deploy via BLE tab sends an existing .cute model package directly to the ESP32. The package is uploaded byte-for-byte; no conversion is performed in this tab.

Figure 3. Deploy via BLE tab.

1	Choose the .cute package to deploy. This can be a base or adapted package.
2	BLE address is optional. Leave it empty to scan automatically for a device named "Cute-YOLO". Enter an address only when you need to target a known device explicitly.
3	The direct deployment pipeline is: .cute -> BLE scan/connect -> upload -> verify -> activate.
4	Press Upload .cute. The ESP32 firmware must already be running and the Python package bleak must be installed.

9. Tab 4 - Log

The Log tab is the authoritative place to check training, adaptation, conversion, packaging, and BLE deployment progress. It is also the easiest place to copy experiment records for a paper or lab notebook.

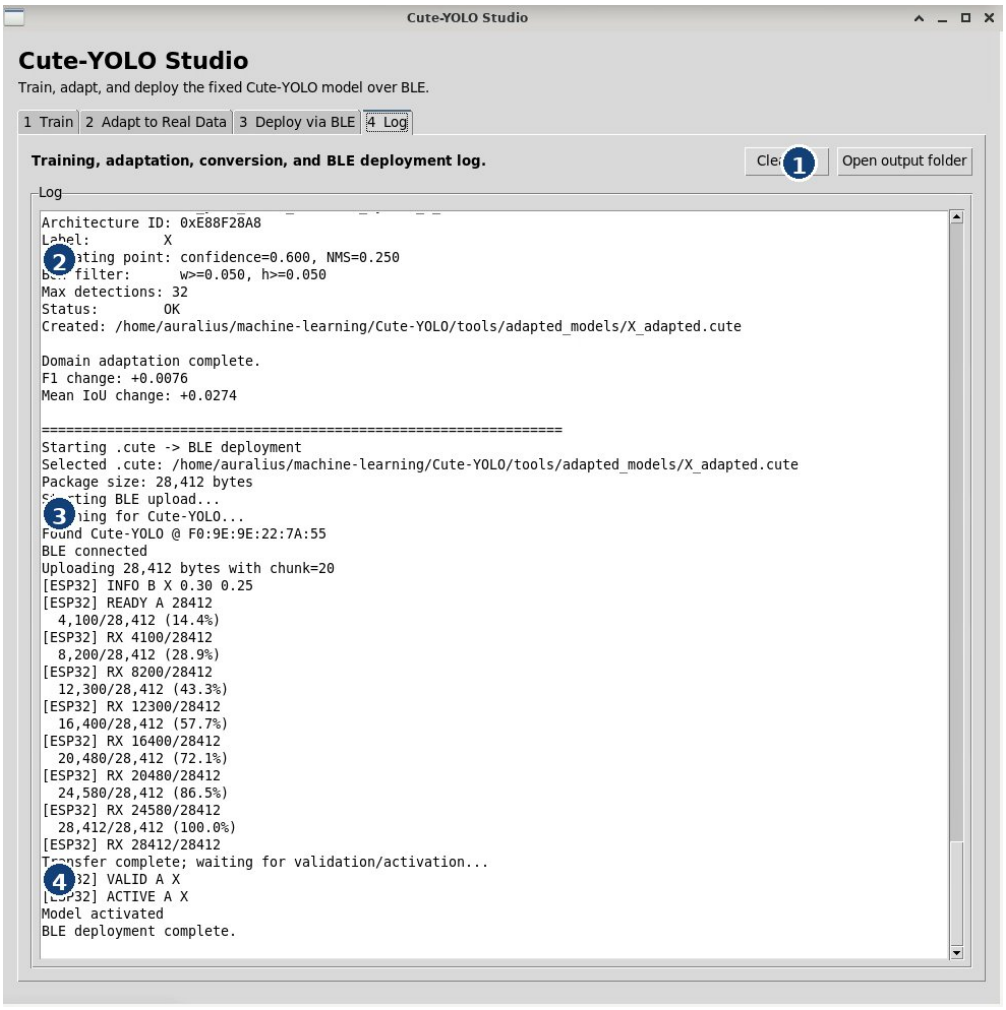


Figure 4. Log tab showing successful package creation and BLE activation.

1	Utility buttons. Clear log removes the current text. Open output folder jumps to the active output directory.
2	Package summary. Check the architecture ID, label, confidence/NMS operating point, box filter, maximum detections, status, and output path.
3	BLE transfer progress. The log reports device discovery, connection, byte counts, and percentage transferred.
4	Validation and activation. A successful deployment ends with messages such as VALID A, ACTIVE A, Model activated, and BLE deployment complete.

10. Recommended End-to-End Workflows

A. Complex / distinctive objects

- Examples: faces, vehicles, animals, or other visually rich single-class targets.
- Use Complex / distinctive preset.
- Keep hard-negative mining ON.
- Keep footprint/halo and crowd-aware weighting OFF unless a specific experiment shows they help.
- Use a practical minimum size (8 px at 128 x 128 is a useful starting point for face-like targets).
- Enable mild horizontal-flip, brightness, contrast, and shuffle augmentation when appropriate.
- Train base model, then inspect FP32 and INT8 validation metrics.
- Collect real ESP32-domain images and adapt if the physical camera domain differs.
- Deploy the INT8-selected .cute package without manually overriding its threshold unless testing an ablation.

B. Primitive / repetitive objects

- Examples: circles, blobs, simple geometric shapes, or repeated nearby objects.
- Use Primitive / repetitive preset.
- Hard-negative mining remains useful.
- Enable box-footprint + halo negatives to suppress objectness inside and immediately around confusing object geometry.
- Enable crowd-aware weighting when neighboring repeated objects form geometric pseudo-objects.
- Choose minimum object size according to the synthetic generator and physical camera target.
- Use the same quantize -> INT8 sweep -> .cute -> BLE deployment path.

Paper / experiment logging: For reproducible experiments, record the selected preset, overlap loss, augmentation switches, minimum object size, source/real mixing ratio, real-validation split, and the final INT8 confidence/NMS operating point.

11. Troubleshooting

Symptom	What to check
Model trains but misses physical objects	Check the real camera domain, minimum-size filtering, augmentation, and the INT8-selected operating point. If needed, collect verified real data and use domain adaptation.
Many false positives	Confirm the task preset. Complex targets benefit from HNM; repeated primitives may need footprint/halo and crowd-aware weighting.
Two nearby objects become one target	The 16 x 16 head can represent one object center per cell. If two centers land in the same cell, a grid-cell collision occurs.
Tiny faces disappear from training	Boxes below the minimum object size are intentionally dropped after crop/resize to 128 x 128.
BLE scan finds no device	Make sure the ESP32 Cute-YOLO firmware is running, BLE is available on the host, and the device advertises as Cute-YOLO. Optionally enter the BLE address.
Upload reaches 100% but model is not active	Read the Log tab for validation/activation messages. Successful transfer alone is not enough; look for VALID and ACTIVE.
INT8 metric differs from FP32	Use the INT8 deployment sweep. Small output changes after quantization can change the optimal confidence or NMS threshold even when localization quality is preserved.

12. Quick Reference

Item	Cute-YOLO Studio behavior
Input	128 x 128 grayscale
Head	16 x 16 x 5
Object classes	Single class per model package
Base training	All layers trainable
Specialized adaptation	Stem + head frozen; Hybrid A/B trainable
Quantization	Strict INT8
Deployment calibration	Confidence + NMS selected on INT8 validation outputs
Package	.cute
Deployment	BLE upload -> verify -> activate
Runtime max detections	32
Example data	dataset_examples/: source/base-training ZIPs and real-domain adaptation ZIPs
TFT display	ST7735 128 x 160 SPI, landscape 160 x 128
TFT SPI pins	SCLK=GPIO39, MOSI=GPIO40, CS=GPIO38, DC=GPIO41, RST=GPIO42, MISO unused
BOOT button	GPIO0 on the current ESP32-S3 DevKitC-1 fixed 8+24 firmware

Detection capacity: A 16 x 16 head has 256 possible object-center cells, but the runtime output is capped at 32 detections. The practical crowded-scene limit is often same-cell center collision rather than the raw 256-cell representation.

End of manual - see dataset_examples/ for ready-to-use workflow examples, Section 3 for the tools/ directory, and Section 4 for ESP32-S3 TFT wiring.